Part 1 — How Agentic AI Works

1. What is Agentic AI?

Regular AI models (like a basic chatbot) just respond to one message at a time. They do not plan, they do not remember what they did before in any meaningful way, and they cannot take actions in the real world. You give them input, they give you output. Done.

Agentic AI is different. An AI agent is a system that can pursue a goal over multiple steps — thinking about what to do, using tools, checking its own work, and continuing until the goal is reached. It is not just generating text. It is actively doing things.

> **Simple version:** A regular LLM answers questions. An AI agent sets goals, makes decisions, takes actions, and keeps going until the job is done.

2. The Core Loop — Perception, Reasoning, Action

Every agentic system runs on a loop. The agent keeps cycling through these steps until it decides the task is complete:

- **Perceive —** Take in information. This could be a user message, a tool result, data from a database, or feedback from the environment.
- **Reason —** Think about what to do next. Most agents use an LLM for this. The model reads the current context and decides the next step.
- **Act —** Execute the decision. Call a tool, write to a file, search the web, call an API, or respond to the user.
- **Observe —** Look at what happened. Did the action work? What did the tool return? Update the plan based on the result.
- **Repeat —** Go back to Reason and decide the next step. Keep looping until the goal is reached.

This is called the ReACT loop (Reason + Act). It is the foundation that almost all agentic systems are built on.

3. The Five Building Blocks of an Agent

i. Planning

The ability to break a complex goal into smaller steps and decide what to do and in what order. Without planning, an agent can only handle simple one-step tasks.

- Chain-of-Thought (CoT): the agent thinks step by step before acting — like writing out your working before answering a math question
- Tree of Thoughts (ToT): the agent explores multiple possible paths and picks the best one
- ReACT: the agent alternates between reasoning and taking actions in tight iterations

ii. Tool Use

Agents can call external tools to get things done in the real world. A tool is basically a function the agent can call — you define what it does and describe it, and the LLM decides when to call it.

Tool Type	Example
Search tool	Search the web or internal knowledge base
Code execution	Run Python to compute results or analyze data

Tool Type	Example
Database query	Read or write to a database
API call	Send emails, post to Slack, trigger a workflow
File operations	Read, write, or edit files
Memory read/write	Store or retrieve information from agent memory

iii. Memory

Memory is how agents remember things. Without it, every step starts from scratch.

- **Short-term (working memory):** the current conversation or task context, held in the LLM's context window. Gone when the session ends.
- **Long-term (external memory):** information stored in a vector database or key-value store. Persists across sessions. Retrieved via semantic search.
- **Episodic memory:** records of past actions the agent took and what happened. Helps it avoid repeating mistakes.
- **Procedural memory:** stored knowledge of how to perform certain tasks — effectively the agent's skill library.

iv. Reflection and Self-Correction

A good agent checks its own work. After taking an action, it evaluates whether the result makes sense and whether it needs to try again differently. This is what separates agents from simple chatbots.

- The agent reviews its output, checks it against the goal, and decides if it needs another step
- Example frameworks: Reflexion, Self-RAG (the agent decides whether it even needs to retrieve before answering)

v. Action Space

The set of things an agent is allowed to do. This is defined by the tools you give it. A narrow action space means a focused, safer agent. A wide action space means a more capable but harder-to-control one.

4. What Happens When You Give an Agent a Task

Here is a step-by-step trace of what happens when you give an agent a task like 'Research our top 3 competitors and write a summary report':

- Agent receives the task and adds it to its working memory
- Planning module breaks it into sub-goals: (1) identify competitors, (2) research each one, (3) write the report
- For sub-goal 1, the agent calls a web search tool with a query
- It reads the results, extracts the competitor names, and stores them in memory
- For sub-goal 2, it loops — calls the search tool for each competitor, reads the output, stores notes
- For sub-goal 3, it retrieves all the notes from memory and calls the LLM to write a structured report
- It reflects on the output — is it complete? Does it cover all three? If not, it loops back and fills the gaps
- Finally it returns the report to the user

> **Key insight:** The agent decides its own next step at every iteration. You do not script each action. You just give it a goal and tools, and it figures out the path.

5. Agentic Frameworks in 2026

These are the main frameworks used to build agents today:

Framework	Made by	Orchestration style	Best For
LangGraph	LangChain	Directed graph (nodes + edges)	Complex multi-step flows, fine-grained control
CrewAI	CrewAI	Role-based crews with tasks	Team-style agents with defined roles
AutoGen	Microsoft	Conversational group chat	Research, multi-agent debates
OpenAI Agents SDK	OpenAI	Handoffs between agents	Production-grade, tight OpenAI integration
Google ADK	Google	Graph + A2A protocol	Cross-framework agent communication
Anthropic Agent SDK	Anthropic	Tool-use + orchestration	Claude-native agentic workflows

Part 2 — Building a Multi-Agent Chatbot

6. What is a Multi-Agent System?

A multi-agent system is where multiple specialized AI agents work together to handle a task. Instead of one agent that tries to do everything, you have a team of agents — each focused on one area — that communicate with each other to produce a final result.

Think of it like a company. There is a manager who breaks work down and assigns it, and there are specialists who each handle their domain. The manager collects the results and delivers the final output to the user.

> **Why not just one big agent?** One agent handling everything becomes slow, expensive, and hard to debug. Splitting by speciality means each agent has a focused context, uses fewer tokens, and is easier to test and improve independently.

7. Multi-Agent Architecture Patterns

Pattern 1 — Supervisor / Orchestrator

One central orchestrator agent receives the user's message, decides which specialist agents to call, passes tasks to them, collects results, and returns a final unified answer.

- Orchestrator breaks the task into sub-tasks
- Each sub-task goes to the right specialist agent (e.g. ResearchAgent, WritingAgent, DataAgent)
- Specialist agents run independently and return results to the orchestrator
- Orchestrator synthesizes everything into a final response
- Best for: most real-world use cases — clear separation of concerns, easy to debug

Pattern 2 — Sequential Pipeline (Chain)

Agents pass the result of one step directly to the next agent in a fixed sequence. Agent A finishes → hands off to Agent B → hands off to Agent C.

- Each agent adds to or transforms the previous agent's output
- Example: DataFetcher → Summarizer → FactChecker → ResponseFormatter
- Best for: document processing pipelines, report generation, structured workflows

Pattern 3 — Swarm / Peer-to-Peer

Agents communicate directly with each other without a central controller. Any agent can hand off to any other agent based on what the task needs.

- More flexible, but harder to control and debug
- OpenAI's Swarm library is an example of this pattern
- Best for: exploratory tasks, research, when the path is unpredictable

Pattern 4 — Hierarchical (Multi-Level)

A top-level orchestrator delegates to mid-level coordinators, who in turn manage their own teams of worker agents. Like a proper organizational chart.

- Best for: very large systems where one orchestrator would be overloaded

- Example: a top-level agent manages a 'Research Team' coordinator and a 'Writing Team' coordinator, each with their own agents

Pattern	Communication	Control	Best For
Supervisor	Hub-and-spoke to orchestrator	High	Most production systems
Sequential pipeline	A → B → C (linear)	High	Fixed workflows, document pipelines
Swarm	Any agent to any agent	Low	Open-ended research tasks
Hierarchical	Multi-level orchestrators	High	Large enterprise systems

---

8. How Agents Actually Communicate

This is the technical part. There are three main ways agents pass information to each other:

Shared State / Shared Memory

All agents read from and write to a shared state object. The orchestrator controls what goes into state and each agent reads what it needs.

- LangGraph uses this approach — state is a typed dictionary that flows through all nodes (agents) in the graph
- Clean and traceable — you can inspect the state at any point in the workflow
- Agents do not need to know about each other — they just read state and write their output back to state

Message Passing

Agents send structured messages to each other, like a group chat. Each message has a role (which agent sent it) and content.

- AutoGen uses this approach — agents participate in a GroupChat and respond to messages from other agents
- Easy to read and reason about — the conversation history shows exactly what was discussed
- Can get expensive: every round-trip involves a full LLM call with accumulated history

Tool Calls / Handoffs

One agent calls another agent as if it were a tool. The calling agent specifies what it wants done, the receiving agent runs and returns a result.

- OpenAI Agents SDK uses this — an agent 'hands off' execution to another agent
- CrewAI also supports this — one agent can delegate a task directly to another
- The cleanest pattern for maintaining focused context — each agent only sees what is relevant to it

9. How to Build a Multi-Agent Chatbot — Step by Step

Here is how you would actually build one. We will use the Supervisor pattern as an example since it is the most common.

Step 1 — Define your agents

Decide what specialists you need. Each agent should have one focused job.

- **OrchestratorAgent:** receives user message, routes to specialists, synthesizes final answer

- **ResearchAgent:** searches the web or knowledge base for information
- **DataAgent:** queries databases, runs analysis, returns structured data
- **WritingAgent:** formats and writes polished responses from raw inputs
- **SafetyAgent:** reviews outputs for policy violations before they reach the user (optional but recommended in production)

Step 2 — Give each agent its tools and system prompt

Each agent needs a system prompt defining its role, and a set of tools it is allowed to use.

- ResearchAgent system prompt: 'You are a research specialist. Your only job is to find accurate information using the search tool. Return your findings in structured bullet points.'
- Give ResearchAgent only the search tool. Do not give it the database tool — that is DataAgent's job.
- Scoping tools tightly per agent keeps each one focused and reduces hallucination

Step 3 — Set up communication

Choose your framework and communication pattern:

- **LangGraph:** define a graph where nodes are agents and edges define who can pass to whom. Use a shared state dict.
- **CrewAI:** define a Crew with agents and tasks. Set process=Process.hierarchical for the supervisor pattern.
- **AutoGen:** create an AssistantAgent per role, a UserProxyAgent for the human, and a GroupChat that manages turn-taking.

Step 4 — Handle context carefully

This is where most multi-agent systems break. Passing the entire conversation history to every agent is expensive and causes confusion.

- Pass only what each agent needs — not the full history
- Use a summary: after each agent finishes, compress its output to a short summary before passing it on
- Aim for 200-500 token context per agent call, not 5,000-20,000 tokens

Step 5 — Add routing logic

The orchestrator needs logic to decide which agent to call. There are two approaches:

- **LLM-based routing:** the orchestrator LLM reads the user message and decides which agent is most appropriate. Flexible but adds an LLM call.
- **Rule-based routing:** simple if-else or keyword matching. Much faster and cheaper. Use for predictable, well-defined workflows.

Step 6 — Add guardrails and fallbacks

- Set a maximum number of agent loops — prevent infinite cycles
- If an agent fails or times out, fall back to a default response rather than crashing
- Log every agent call: which agent ran, what it received, what it returned, how long it took
- For user-facing products, always have a SafetyAgent or output filter as the final step before returning to the user

10. Framework Comparison for Multi-Agent Chatbots

Framework	Communication style	Complexity	Production-ready	Best pick when...
LangGraph	Shared state graph	Medium-High	Yes	You need fine-grained control and complex conditional flows

Framework	Communication style	Complexity	Production-ready	Best pick when...
CrewAI	Role-based task delegation	Low-Medium	Yes	You want fast setup with clear agent roles
AutoGen	GroupChat messages	Medium	Partial	You are doing research or multi-agent reasoning tasks
OpenAI Agents SDK	Handoffs	Low	Yes	You are using OpenAI models and want simplicity

> **Best practice for 2026:** Start with CrewAI or OpenAI Agents SDK for quick results. Move to LangGraph when you need precise control over state and conditional routing in production.

---

11. Common Mistakes in Multi-Agent Systems

- **Passing full conversation history to every agent —** massively increases token cost and confuses agents. Always pass summarized, scoped context.
- **No loop limit —** agents can get stuck calling each other endlessly. Always set a max_iterations cap.
- **No fallback —** if one agent fails, the whole system crashes. Build fallback responses at the orchestrator level.
- **Overlapping agent responsibilities —** two agents with similar jobs creates conflicts. Keep roles strictly separate.
- **No observability —** if you cannot see what each agent did and why, debugging is impossible. Log every step.
- **Ignoring latency —** each agent call adds time. A 5-agent pipeline where each agent takes 2 seconds is a 10-second response. Plan accordingly.